

Performance Evaluation of Reinforcement Learning and Genetic Algorithms

Kanav Bengani bengani.k@northeastern.edu

Nathan Yan yan.na@northeastern.edu

Ryan Saperstein saperstein.r@northeastern.edu

Nicholas Pfeiffer pfeiffer.n@northeastern.edu

Northeastern University, Boston, MA

December 4, 2024

Abstract

We wanted to test the capabilities and learning rates as well as strategies of reinforcement learning and genetic algorithms within an agent vs agent tank game. Our goal was to discover which agent (RL or GA) could create a 'better' winning policy and then identify which of these policies performs better when placed against each other. Through respective rewards and fitness functions, we trained these agents against an optimal agent and once these agents were believed to have been fully trained, noted the results and observed what actions they performed against one another.

Introduction

In this project, two agents are trained to compete in a game, where one uses a genetic algorithm to select its move each turn and the other uses a reinforcement learning algorithm to select its move each turn. The game itself consists of the two agents taking turns moving on a board. The game ends when one agent successfully shoots the other.

The motivation behind this project is to see how a genetic algorithm approach compares to a reinforcement learning approach when it comes to how quickly an agent can learn to play a game. This project compares the two approaches in a very controlled environment to see which approach can learn to play the game effectively more quickly. While there are many instances of agents being trained to play games with either genetic algorithms or reinforcement learning, there is little existing literature comparing the two approaches. Thus, this document seeks to fill that gap and compare the two approaches.

Related Work

In 2023, Robert Gold et al used a genetic algorithm to play the game *Bombberman*, which is similar to the game that we designed. Similarly, in 2021, Hung and Triet trained an agent with reinforcement learning algorithm to play *Bombberman*. While our approach is somewhat inspired by Gold et al and Hung and Triet, they both only sought to design a single agent that could effectively play *Bombberman*, while we seek to perform a comparison across two different agents.

Also in 2023, Uthayasuriyan et al performed a comparative study on genetic algorithms and reinforcement learning with regards to solving the traveling salesman problem. While similar in aim, our project is much more controlled in scope and therefore should produce a more robust comparison than on such an open-ended problem. Similarly in 2019, Xiang performed a comparative study on genetic algorithms and reinforcement learning with regards to autonomous driving; again, since our project is much more controlled in scope, it should provide a more robust comparison of these two methods.

Methods

One agent was trained using reinforcement learning, specifically q-learning. At a high-level, with each action the agent took, it received a positive or negative reward. This slowly led to the agent being able to learn which actions are more optimal for it be able to win. Each value in the q-table was computed using the following equation, where s and a represent the state-action pair the agent took, η represents the number of updates for the given state-action pair, s' represents the state after the opponent has played a move, a' represents the possible actions to take at s' , and γ represents the discount factor (chosen to be .9 in our case):

$$Q_{new}(s, a) = (1 - \eta) * Q_{old}(s, a) + \eta(R(s, a) + \gamma * \max_{a' \in actions} (Q_{old}(s', a')))$$

On the other hand, for an agent trained using genetic algorithm, a given state initially was a randomly-set q table (policy) where each state had a random action set to 1 (and 0 for all other actions). Each policy was run for a full episode. At each turn, a reward was appended for that given episode. After an arbitrary amount of episodes n had passed, the policies were ranked based on the reward they accumulated in their respective episodes. Badly ranked policies were eliminated, while well-ranked policies were combined and mutated to produce more policies.

Data & Experiments

1 Environment

The environment in which the agents played was a $n \times n$ grid in which agents start diagonally from one another. The board contains walls which cannot be broken or passed through; all other tiles on the board can be passed through and shot through unless it already holds a character. An agent wins the game by shooting its opponent a single time. Agents have the ability to move up, down, left, or right, rotate their gun left or right, or shoot. Agents can only shoot in the direction that their gun is facing. The environment (besides the enemy's direction) is fully known to both agents.

Initially, we wanted to include additional agent property variables, such as speed, ammo capacity, fuel, and breaking or building obstacles. The learning models would then attempt not only to pick an optimal action, but also to pick an optimal set of properties in order to win the game. However, due to an astronomically high sample space, we chose to simplify our representation and perform an experiment that was more attainable.

2 Implementing the Models

2.1 Implementing Reinforcement Learning

As mentioned above, the reinforcement learning agent was built using Q-learning. Each state in the q table was a combination of various observations extracted from the board from the perspective of the agent. These observations include the current agent's position, direction, and the opponent agent's position. The opponent agent's direction was not included for simplicity reasons. Each state in the q-table was mapped to the list of 7 actions that is mentioned in the 'Environment' section. Once a move had been made by the opponent agent, the board state was ingested by the RL agent to actually place the value in the q-table given the state-action pair it had taken along with what the new state of the board looked like (to compute rewards and $Q_{old}(s', a')$); this value specifically was calculated by the equation outlined in the 'Methods' section. The reward at each turn was calculated using the following heuristics:

1. +1000000: Shot the opponent
2. -1000000: Got shot by the opponent
3. -500: Missing a shot
4. -100: None of the above was accomplished

Other heuristics such as manhattan distance and facing the opponent were included initially in our implementation. However, we chose to remove this. This is because after training, it seemed to end up farming actions (leading to a closer manhattan distance or facing the opponent more) for points rather than actually trying to win. After a high number of episodes, the values in the q-table would start to converge. More details on the training of the RL agent can be found in 'Experiment 1'.

2.2 Implementing Genetic Algorithm

The genetic algorithm agent was built using 9 initially randomly-set policies with each state having a paired action. For each of the 9 agents, we calculated the fitness based on its actions at different states. The fitness was evaluated using the following heuristics:

1. +1000000: Shot the opponent
2. -1000000: Got shot by the opponent
3. +100: A closer manhattan distance to the opponent
4. +100: Facing the opponent more (using vector dot products)
5. -500: Missing a shot
6. -100: None of the above was accomplished

Once all policies had run through each of the 4 starting game states (agent at each of the 4 different corners of the board), we would maintain the best four policies and create children based on random policy pairs. A child was created through both combination and mutation. Combinations between two parent policies were executed by picking the action for a given state that resulted in a higher reward between the two parent policies. This state-action pair was reflected in the child policy. Then, with a small mutation probability, the action would be replaced with another a random action. Through evolution and natural selection by the above-mentioned means, the states would naturally converge to a more optimal policy that can beat its opponent. More details on the final-version of GA training can be found in 'Experiment 3'.

2.3 Building the Optimal Agent

In order to train both our reinforcement learning and genetic algorithm models, we needed to design an agent that these training models can consistently learn against. For this agent, we trained two reinforcement agents against each other for a few million episodes. We started both agents with a very high decay rate to promote a large exploration of states. Using this method, over time the agents were able to learn viable strategies as their action choices gradually became non-random. Once we were able to produce a fairly effective optimal agent, we stored the trained agent's Q-Table.

3 Experiments

3.1 Training an RL Agent

To train the RL agent, we experimented with both an RL agent trained against the optimal agent's Q table using low decay value of (0.9995) and a high decay value of (0.999995) in order to see if there would be any significant strategical differences between an RL agent that has explored significantly more states than the other. We considered the RL agent to be fully trained when they reached a specific win threshold against the optimal agent's Q table policy within the past hundred episodes. In order to stop training, we determined a good point would be when the agent won ninety of the past one hundred episodes. We then stored the finalized Q-Table for the low and high decay agents.

3.2 Training a GA Agent (Initial)

Like the RL agent, we trained the GA agent against the optimal agent's Q-Table, which allowed the GA agent to learn in a consistent environment. We started with a random set of 8 policies, which we alternated after every game. For each game, the agent would determine it's next action based on the policy for that state. For unexplored states, we would assign a random action.

In order to merge and repopulate policies, we evaluated each policy's fitness level. We determined fitness to be the number of moves the agent took to win or lose. We prioritized policies such that winning fast is prioritized over winning slow, winning slow is prioritized over losing slow, and losing slow is prioritized over losing fast.

Then, we kept the best 4 policies, removed the worst 4 policies, and merged the best 4 policies to produce 4 children. To create children, we randomly selected policies from the top 4 and randomly merged their states together.

Additionally, we included the possibility of random mutations occurring. Every state within the policy had a small chance of mutating, so that the agent could still explore new possibilities with the new random action.

3.3 Training a GA Agent (Final)

We realized that there was an issue with a policy that based fitness solely on the number of turns as it would also consider games where GA avoided dying to the optimal agent and tied to have a good fitness and wanted to limit the randomness of GA's state-action pairs. We switched approaches to implementing the GA model that calculates fitness as a reward based on actions taken and the result of the game. This way, state-action pairs chosen during the combination process would be less random and more beneficial, as the chosen state-action pair would be based on whichever parent policy had a higher fitness at the state. The mutation rate remained the same from the initial version to the final version as well as the initial randomization of policy's state-action pairs. Once again we trained the GA agent

against the optimal agent until a certain win threshold was reached (the same as RL’s win threshold). After training, we stored the best policy found by the GA agent based on its fitness.

3.4 RL agent vs GA agent

After the training of both RL agents at a low and high decay as well as the GA agent (from experiments 1 and 3 respectively), we wanted to view the results of facing these agents against each other with training disabled. To run the RL agents against the GA agent, we used the best policy found by the GA agent (based on fitness value) and viewed the result (win, lose, tie) of an episode at each of the 4 starting configurations.

Results

3.5 Results of Training an RL Agent

Our Low Decay agent took on average 4,566 episodes of training to consistently beat the optimal agent 90% of the time. During this time, the agent explored 511 unique states.

Our high decay agent took on average 317,843 episodes of training to consistently beat the optimal agent 90% of the time. During this time, the agent explored 849 unique states.

So, both agents were trained to consistently beat the optimal agent, but the high decay agent did so while exploring more states.

The low decay agent was found to have taken a very defensive strategy, staying within one or two moves from its starting position and rotating or shooting to stall until the optimal agent approached line of sight and was shot at. The high decay agent appeared to take a slightly more aggressive approach than the low decay agent, moving towards the optimal agent and then slightly stalling until the optimal agent approached line of sight, causing games to end quicker with less turns taken overall. This is attributed to the variety of states it could observe due to its higher value of decay and therefore randomness.

3.6 Results of the Final Version of GA agent training

The GA agent took approximately 3,085 episodes to consistently beat the optimal agent 90% of the time. We found that it took a strategy that was in between the passive defensive qualities of the low decay RL agent and the more aggressive high decay RL agent. It would move further from its starting designation towards the optimal agent and then stall until victory, however it was not as efficient nor aggressive as the high decay RL agent.

3.7 Results of running an RL agent vs GA agent

When the low decay RL agent and the GA agent faced each other, both resorted to extremely defensive strategies. They refused to budge from their initial positions, resulting in a game of repetition until the game was called off as a tie. Similarly when we paired the high decay RL agent against the GA agent, generally both maintained their defensive strategies, hoping that the other would be an aggressor and close the distance. Interestingly, depending on the starting configuration, the high decay RL agent would act as the aggressor and close the distance between itself and the GA agent, however this generally led to the GA agent’s victory. We believe this behavior was due to the extra explored states that the high decay RL agent experienced, giving it more variety in its strategy and opening up a path for aggression. However, we believe that it wasn’t able to optimize this strategy and failed against the GA agent due to its defensive approach against its training opponent (the optimal agent) being already overwhelmingly successful, leaving the high decay RL agent without the need to improve its aggressive strategy.

Conclusion

This project explored the performance of reinforcement learning (RL) and genetic algorithms (GA) by comparing their ability to develop winning strategies in a controlled environment. Our findings reveal key insights into the relative strengths of these two approaches. RL, specifically Q-learning, demonstrated significant differences between low-decay and high-decay learning rates in terms of convergence, exploration, and strategy diversity. The GA agent, through evolving policies and natural selection,

displayed similar convergence in comparison to RL low-decay but a lot faster in comparison to RL high-decay, with seemingly better results when paired to fight against each other. Despite differences in the underlying mechanisms, both approaches succeeded in training agents capable of consistently defeating the optimal agent. This comparative study provides valuable insights for future applications of RL and GA in competitive and dynamic environments, with potential extensions to more complex tasks and hybrid approaches that combine the strengths of both methodologies. It would be interesting if other works built off of our work and attempted to see the results of training the RL and GA agents against a defensive optimal agent, encouraging offensive strategies to be built by RL and GA, or finding a good middle ground strategy of defense and offense.

GitHub

<https://github.com/kanavbengani/reinforcement-learning-vs-genetic-algorithms>

Team Contributions

1. Nick Pfeiffer: Setting up the environment (pygame, agent states, actions, characters, board).
2. Ryan Saperstein: Implementing the reinforcement learning agent.
3. Kanav Bengani: Responsible for the implementing the genetic algorithm agent. This includes the logic for running through different policies each episode, calculating the fitness of each policy as well as handling combinations and mutations every 9 episodes.
4. Nathan Yan: Training of RL, GA, and optimal agents and executing experiments.

Throughout the project, each of us ended up contributing to other parts of the project as and when needed. The above simply ended up being what each person was responsible for.

References

- [1] Gold, Robert, et al. "Genetic Programming and Coevolution to Play the *Bomberman* Video Game." *Lecture Notes in Computer Science*, 9 April 2023, pp. 765–779, https://doi.org/10.1007/978-3-031-30229-9_49.
- [2] Hung, Ngo, and Minh Triet. Intelligent Bomberman with Reinforcement Learning, 2021. [essay.utwente.nl/87030/1/Ngo_BA_EEMCS.pdf](https://utwente.nl/87030/1/Ngo_BA_EEMCS.pdf).
- [3] Uthayasuriyan, Agash, et al. "A comparative study on genetic algorithm and reinforcement learning to solve the traveling salesman problem." *Research Reports on Computer Science*, 2 May 2023, pp. 1–12, <https://doi.org/10.37256/rrcs.2320232642>.
- [4] Xiang, Ziyi. A Comparison of Genetic Algorithm and Reinforcement Learning for Autonomous Driving, 2019, www.diva-portal.org/smash/get/diva2:1358693/FULLTEXT01.pdf.
- [5] <https://github.com/Forestf90/Bomberman/tree/master>: This github of the game bomberman served as a basis for how we could create our 2D tank game using pygame.